

Graph Closure Extraction Strategies: Deep Architecture Guide

The Problem We're Solving

In the Vedic Math Speed Gym architecture, your **Neo4j AuraDB Free tier** stores the prerequisite graph (`:Technique` nodes connected by `[ :PREREQUISITE_OF ]` relationships). As you scale from hundreds to tens of thousands of densely interconnected nodes, you need to precompute transitive closures — answering "what does this student need to know before learning X?".

The naive approach (`[ :PREREQUISITE_OF*1..5 ]`) triggers a **Cartesian explosion** on dense graphs. Each variable-length traversal fans out exponentially, and AuraDB's strict JVM heap limits (Free tier) will crash with `OutOfMemoryError` or transaction timeouts.

The four strategies below solve this at different scales, with Strategy A being the immediate fix and Strategy D being the enterprise-grade long-term solution.

Strategy A: APOC BFS Expander (Neo4j-Side Optimization)

What It Is

The **APOC (Awesome Procedures On Cypher)** library replaces Neo4j's default **Depth-First Search (DFS)** with **Breadth-First Search (BFS)**. On dense graphs, DFS wastes memory by diving deep into branches before exploring siblings. BFS explores level-by-level, which is more memory-efficient and produces deterministic results.

The Query

```

MATCH (start:Technique)
CALL apoc.path.spanningTree(start, {
  relationshipFilter: "PREREQUISITE_OF>",
  minLevel: 1,
  maxLevel: 5,
  limit: 1000          -- Safety cap per starting node
})
YIELD path
RETURN
  start.content_hash AS ancestor,
  last(nodes(path)).content_hash AS descendant,
  length(path) AS depth

```

Why It Works

- `spanningTree` guarantees **no cycles** — critical for prerequisite graphs where circular dependencies would loop forever
- `limit: 1000` caps the explosion per starting node
- BFS uses a **queue** (FIFO) instead of a **stack** (LIFO), so memory footprint is bounded by the maximum frontier width, not depth

AuraDB Free Tier Considerations

Factor	Impact
APOC availability	✅ Built into AuraDB — no plugin installation needed
Heap pressure	🟡 Moderate — BFS is better than DFS but still in-memory
Execution time	🟡 ~2-5 seconds per 1,000 starting nodes at depth 5
When it fails	🔴 When graph exceeds ~15,000 nodes with avg degree >8

When to Use It

Immediate implementation (Week 2). If your graph is under 15,000 nodes and you're on AuraDB Free, this is the zero-infrastructure-change fix. Your

`Neo4jDesignTimeCompiler` Python script just swaps the Cypher query.

Strategy B: Topic-Partitioned Extraction (Batching)

What It Is

Instead of asking Neo4j for the **entire graph's closure at once** (which is what crashes), your Python sync pipeline loops through `taxonomy.yaml` topics and queries **one subgraph at a time**. Each topic (e.g., "VedicMath", "NumberTheory", "Algebra") becomes an independent batch.

The Pipeline Flow

```
# Python: Neo4jDesignTimeCompiler.topic_partitioned_sync()
topics = ['VedicMath', 'NumberTheory', 'Algebra', 'Geometry']

for topic in topics:
    result = neo4j.run("""
        MATCH (a:Technique {topic: $topic})-[:PREREQUISITE_OF
*1..5]->(b:Technique)
        RETURN a.content_hash, b.content_hash, length(path) A
S depth
        """, topic=topic)

    # Batch insert into Postgres
    psycopg2.extras.execute_values(
        cursor,
        "INSERT INTO prerequisite_closure VALUES %s",
        result
    )

    # Commit per topic to avoid giant transactions
    conn.commit()
```

Why It Works

- Each topic's subgraph is **10-50× smaller** than the full graph

- Postgres commits are **per-topic**, so a failure in "Algebra" doesn't rollback "VedicMath"
- Network payload is chunked — no single multi-megabyte response
- Easy to **parallelize** with `concurrent.futures.ThreadPoolExecutor`

AuraDB Free Tier Considerations

Factor	Impact
Heap pressure	● Low — each query touches a small subgraph
Execution time	● Slower overall (~30-60 seconds for all topics)
Network round trips	● Higher — N topics = N queries
Complexity	● Medium — requires topic metadata on nodes

When to Use It

When AuraDB timeouts start occurring with Strategy A. This is your "circuit breaker" — when the global query fails, fall back to topic batches. It requires that your `:Technique` nodes have a `topic` property (which they should from your `taxonomy.yaml` ingestion).

Strategy C: Postgres Recursive CTE (The Architecture Shift)

What It Is

Neo4j is brilliant at **finding paths** but terrible at **exporting massive datasets**. Postgres is the opposite — it excels at relational math on disk. Strategy C splits the work:

1. **Neo4j only exports Depth-1 (direct edges)** — trivial, fast, tiny payload
2. **Postgres computes Depth 2-5 via `WITH RECURSIVE`** — uses disk, not RAM; scales to millions of rows

This is the **"Ledger vs GPS"** principle in action: Neo4j gives you the map (structure), Postgres does the heavy math (closure computation).

Step 1: Neo4j L1 Export (Lightweight)

```
// Extract ONLY direct prerequisite relationships
MATCH (a:Technique)-[:PREREQUISITE_OF]->(b:Technique)
RETURN a.content_hash AS ancestor_hash,
       b.content_hash AS descendant_hash,
       1 AS depth
```

Step 2: Postgres Staging Table

```
CREATE TABLE stg_prerequisites (
  ancestor_hash VARCHAR(64) NOT NULL,
  descendant_hash VARCHAR(64) NOT NULL,
  depth INT NOT NULL,
  PRIMARY KEY (ancestor_hash, descendant_hash)
);
```

Step 3: Recursive CTE Materialization

```
WITH RECURSIVE closure AS (
  -- Base case: L1 links from Neo4j
  SELECT ancestor_hash, descendant_hash, 1 AS depth
  FROM stg_prerequisites

  UNION ALL

  -- Recursive case: join paths up to depth 5
  SELECT
    c.ancestor_hash,
    s.descendant_hash,
    c.depth + 1
  FROM closure c
  JOIN stg_prerequisites s
    ON c.descendant_hash = s.ancestor_hash
```

```

WHERE c.depth < 5
)
INSERT INTO prerequisite_closure (ancestor_id, descendant_id,
depth)
SELECT
    ancestor_hash,
    descendant_hash,
    MIN(depth) -- Keep shortest path if multiple exist
FROM closure
GROUP BY ancestor_hash, descendant_hash;

```

Why It Works

- Postgres executes this on **disk** via temp tables — no JVM heap limits
- `MIN(depth)` automatically handles multiple paths (keeps the shortest prerequisite chain)
- `UNION ALL` with `WHERE c.depth < 5` guarantees termination
- Can handle **millions of rows** without breaking a sweat

AuraDB Free Tier Considerations

Factor	Impact
Heap pressure	● Zero — Neo4j only exports L1 edges
Execution time	● Fast — L1 export is ~100ms; Postgres CTE is ~1-2s
Network payload	● Tiny — only direct edges
Complexity	● High — shifts business logic to Postgres SQL

When to Use It

Phase 3+ (10,000+ nodes / cross-category routing). This is the enterprise standard. The trade-off is that your prerequisite logic now lives partly in SQL, not just Cypher. But for your \$12 Droplet with Postgres, this is infinitely more scalable than Neo4j closures.

Strategy D: Incremental Delta Sync (The Holy Grail)

What It Is

Instead of recomputing the **entire closure** on every sync (which Strategies A-C all do), Strategy D only recomputes for nodes that changed since the last sync. It uses **temporal bookkeeping** to track "dirty" nodes.

The Mechanism

Neo4j Side — Add `updated_at` to every node:

```
MATCH (t:Technique)
SET t.updated_at = datetime()
```

Python Sync Pipeline — Query only changed nodes:

```
last_sync = get_last_sync_timestamp() # From sync_manifest table

changed_nodes = neo4j.run("""
    MATCH (t:Technique)
    WHERE t.updated_at > datetime($last_sync)
    RETURN t.content_hash AS changed_hash
""", last_sync=last_sync)

for node in changed_nodes:
    # Recompute closure ONLY for this node's downstream paths
    subgraph = neo4j.run("""
        MATCH (start:Technique {content_hash: $hash})
        CALL apoc.path.spanningTree(start, {
            relationshipFilter: "PREREQUISITE_OF>",
            maxLevel: 5
        })
        YIELD path
```

```

    RETURN ...
    """ , hash=node[ 'changed_hash' ])

# Upsert into Postgres (update existing, insert new)
upsert_prerequisite_closure(subgraph)

```

Why It Works

- On a typical day, only **1-5% of nodes change** (new content added, difficulty adjusted)
- Recomputing 5% of closures vs 100% = **20× speedup**
- `updated_at` triggers in Neo4j can auto-set timestamps on CREATE/SET
- `sync_manifest` tracks `completed_at` for idempotency

AuraDB Free Tier Considerations

Factor	Impact
Heap pressure	🟢 Minimal — only touches changed subgraphs
Execution time	🟢 Fastest — seconds instead of minutes
Complexity	🔴 Hardest — requires trigger management, partial invalidation logic
Risk	🟡 If a root node changes, entire subtree must recompute

When to Use It

Phase 4+ (daily syncs taking >10 minutes). Do not attempt until you've proven the graph is stable and your sync pipeline is mature. The partial invalidation logic ("if node X changed, which closure rows are now stale?") is the hard part.

Benchmarking Protocol

To safely compare these strategies on production data without breaking the live app, use your existing `sync_manifest` infrastructure:

Step 1: Create Test Tables


```

CREATE TABLE prerequisite_closure_test AS
SELECT * FROM prerequisite_closure WHERE 1=0;

CREATE TABLE sync_manifest (
    id SERIAL PRIMARY KEY,
    source VARCHAR(100),           -- e.g., 'neo4j_apoc_test'
    target_table VARCHAR(100),     -- 'prerequisite_closure_t
est'
    status VARCHAR(20),           -- 'dry_run' | 'productio
n'
    started_at TIMESTAMP,
    completed_at TIMESTAMP,
    record_count INT,
    execution_time_ms INT,
    memory_peak_mb DECIMAL(10,2),
    error_message TEXT
);

```

Step 2: Run Each Strategy

```

# benchmark_closure_strategies.py
strategies = ['apoc_bfs', 'topic_partitioned', 'postgres_cte', 'incremental_delta']

for strategy in strategies:
    started = time.time()

    try:
        if strategy == 'apoc_bfs':
            run_apoc_strategy(target='prerequisite_closure_test')
        elif strategy == 'topic_partitioned':
            run_partitioned_strategy(target='prerequisite_closure_test')

```

```

elif strategy == 'postgres_cte':
    run_cte_strategy(target='prerequisite_closure_test')

elif strategy == 'incremental_delta':
    run_delta_strategy(target='prerequisite_closure_test')

# Log success
log_sync_manifest(
    source=f'neo4j_{strategy}_test',
    target_table='prerequisite_closure_test',
    status='dry_run',
    record_count=get_row_count(),
    execution_time_ms=int((time.time() - started) * 1000)
)

except Exception as e:
    # Log failure without crashing pipeline
    log_sync_manifest(
        source=f'neo4j_{strategy}_test',
        status='dry_run_failed',
        error_message=str(e)
    )

```

Step 3: Decision Metrics

Metric	How to Measure	Good Threshold
AuraDB Heap Pressure	AuraDB dashboard / logs	<70% peak JVM heap
Execution Time	Python <code>time.perf_counter()</code>	<30 seconds total
Network Payload	Size of Neo4j result sets	<5MB per batch
Postgres Ingestion Time	<code>COPY</code> or <code>execute_values</code> duration	<5 seconds

Metric	How to Measure	Good Threshold
Code Complexity	Lines of Python + SQL	Subjective — team familiarity

Decision Matrix

Condition	Recommended Strategy	Rationale
Graph < 15K nodes, AuraDB Free	Strategy A (APOC BFS)	Zero infrastructure change, built-in AuraDB support, handles current scale
Global queries timeout, topic metadata exists	Strategy B (Topic-Partitioned)	Prevents timeouts, leverages existing taxonomy, easy parallelization
Graph > 50K nodes, cross-category bridges	Strategy C (Postgres CTE)	Moves heavy math to Postgres disk, infinitely scalable, enterprise standard
Daily sync > 10 min, graph is stable	Strategy D (Incremental Delta)	20× speedup, only viable after graph matures and triggers are reliable
Mixed conditions (some topics dense, some sparse)	Hybrid: B + C	Use B for sparse topics (fast), C for dense topics (scalable)

My Architect's Recommendation for Vedic Math Speed Gym

Phase	Timeline	Strategy	Action
Phase 1 (Launch)	Weeks 1-15	A	Implement APOC BFS in <code>Neo4jDesignTimeCompiler</code> . Add <code>limit: 1000</code> safety cap.
Phase 2 (Scale)	Weeks 16-21	A → B	When timeouts occur, switch to topic-partitioned. Add <code>topic</code> property to all <code>:Technique</code> nodes.

Phase	Timeline	Strategy	Action
Phase 3 (Cross-Subject)	Month 4-6	C	Implement Postgres Recursive CTE. Create <code>stg_prerequisites</code> table. Migrate closure logic to SQL.
Phase 4 (Mature)	Month 7+	D	Add <code>updated_at</code> triggers. Implement incremental delta sync. Benchmark weekly.